ФСП Таланты — платформа обратного подбора ИТ-специалистов

Документ для сдачи MVP. Единое сопроводительное описание решения: архитектура, механика тестирования и подбора, валидация, интеграция с ФСП, API, библиотеки с версиями и пошаговый запуск. Подробности по темам — в отдельных документах репозитория (список в разделе 19), но для понимания решения они не требуются.

1. Краткое описание и статус

Что это. Веб-сервис с двумя кабинетами — «Кандидат» и «Работодатель». Категория кандидата (специализация × грейд) определяется не резюме, а обязательным путём: опрос → выбор грейда → **тест с серверной проверкой**. Работодатель описывает потребность, получает подборку подтверждённых кандидатов с объяснением и сам направляет приглашение с обязательной зарплатной вилкой — без публикации вакансии. Контакты кандидата раскрываются только той компании, чьё приглашение он принял. Достижения Федерации спортивного программирования (ФСП) — дополнительное преимущество в ранжировании, а не условие доступа.

Статус. Прототип для технической демонстрации; все данные синтетические; интеграция с ФСП — демонстрационная; внешние AI/LLM API не используются. Решение **не** готово к промышленной эксплуатации, **не** проверялось на соответствие законодательству о персональных данных и **не** доказывает точность оценки квалификации на реальных людях.

Что реализовано и проверено (фактические результаты — раздел 11):

- оба кабинета, регистрация с подтверждением email, серверные сессии, CSRF, роли;
- 4 специализации × 3 грейда; банк из 24 параметризованных семейств заданий (72 пары «семейство × грейд»), закрытые ключи на сервере, пересдачи 24 ч / 90 дней;
- подбор по потребности с жёсткими условиями, формулой, объяснением, сохранёнными подборками, уточнениями и закладками; банк кандидатов с фильтрами;
- приглашения с обязательной вилкой GROSS, атомарными статусами и срок 14 дней; раскрытие контактов только принявшей компании; PDF-профиль по тому же презентеру;
- демонстрационный провайдер ФСП; кандидат без ФСП проходит весь путь;
- синтетические данные (114 кандидатов, 4 компании), валидация качества с baseline, OpenAPI (59 операций), Docker Compose, 319 unit-, 150 интеграционных и 5 сквозных тестов.

Что не реализовано (необязательно по ТЗ или вне MVP): загрузка PDF-резюме с распознаванием, регулярные задания, вакансии и отклики, проверка компаний и защита от фиктивных предложений, интеграция с ATS, мобильная адаптация, реальный ФСП ID (OIDC/Keycloak). Концепции этих направлений описаны в разделе 17 и не выдаются за реализованные.

2. Задача и предложенный подход

ТЗ требует платформу с **обратной механикой**: соискатели автоматически распределяются по категориям на основе объективно подтверждаемого профиля, а работодатель сам находит категорию

и выходит на конкретного кандидата с предложением и зарплатой. Главные инженерные вопросы — как подтверждать квалификацию так, чтобы задания нельзя было «раздать», как ранжировать и объяснять выдачу и как доказать, что это работает, не имея эталонных данных.

Вопрос	Решение
Как определить категорию без резюме?	Обязательный путь: опрос → заявленный грейд → тест; категория = **подтверждённая** квалификация по специализации. Заявленный грейд — только самооценка
Как не допустить распространения заданий?	Банк — не набор вопросов, а **семейства-генераторы**: уникальный экземпляр на попытку из (семейство, версия, грейд, seed), закрытые серверные ключи, неизменяемый снимок экземпляра
Нейросеть для генерации?	Нет: она не гарантирует сопоставимую сложность и объективность. Детерминированные генераторы дают контролируемую сложность и воспроизводимость; оценка — код, а не модель
Как ранжировать и объяснять?	Жёсткие условия по подтверждённым данным → формула `70·предпочтения + 25·качество + 5·ФСП` → стабильная сортировка → структурированное объяснение
Как не наказывать за отсутствие ФСП?	ФСП — бонус ≤ 5 из 100, не обходит условия; отсутствие истории — штатный случай
Как раскрывать контакты?	Только после принятия приглашения и только принявшей компании; единый презентер для JSON и PDF; отзыв доступа
Как проверять качество без эталона?	Собственные синтетические данные и разметка, baseline, ручные контрольные примеры, симуляции; ограничения названы (раздел 10)

3. Функциональность по ролям

Кандидат

- **Регистрация** по email с подтверждением (письмо со ссылкой; подтверждает явный POST). До подтверждения доступны только экран подтверждения, повторная отправка и выход.
- **Опрос и выбор специализации** (отрасли, интересы, недавние задачи, опыт) → рекомендация специализации и уровня; выбор активной специализации и заявленного грейда.
- **Профиль:** ФИО, город, часовой пояс, опыт, стек из справочника (с длительностью), soft skills, роли, формат работы, зарплатные ожидания, контакты (email, телефон, мессенджер), ссылки.
- **Тестирование:** выбор уровня, подтверждение старта, 6 мини-кейсов за 30 минут с серверным таймером и автосохранением, завершение или прерывание; результат: баллы групп, статус, подтверждённые и неподтверждённые навыки, рекомендации; история попыток. Допуск показывает причину отказа и **точную дату** следующей попытки.
- **Категория:** подтверждённая квалификация и неизменяемая история подтверждений.
- **Приглашения:** список и карточка с зарплатой и условиями **до принятия**, перечень раскрываемых данных, принять/отклонить; компании с доступом к контактам и их отзыв.
- **Приватность и ФСП:** согласие на публикацию (с историей), видимость в поиске, пауза, скрытие ожиданий и достижений ФСП, каналы раскрытия, предпросмотр обезличенной карточки, привязка ФСП ID, PDF-профиль.

Работодатель

- **Компания:** название, отрасль, сайт, описание, контактное лицо и способ связи (данные для приглашения).
- **Потребность:** название, команда и задачи, специализация, допустимые грейды, обязательные и желательные навыки (режимы `VERIFIED_ONLY` / `SELF_REPORTED_ALLOWED`), формат работы, город, **зарплатная вилка**. Обязательный навык без проверяющих заданий отклоняется (`422`). Правка создаёт новую версию; «Тест под требования» показывает покрытие требований заданиями.
- **Подборка:** кандидаты с местом, баллом, разложением по компонентам, «Почему подходит» / «Что пока не подтверждено»; уточнения (только с ФСП, минимальный балл, дополнительные навыки, города) создают дочернюю подборку без потери исходной; история подборок; пустая подборка объясняет причины отсева.
- **Банк кандидатов:** все опубликованные кандидаты с фильтрами (специализация, грейд, категория, навыки, формат, город, наличие достижений ФСП); закладки.
- **Карточка кандидата:** обезличенная до принятия, с контактами — после; PDF по тем же правам.
- **Приглашения:** создание с заголовком, описанием, вилкой GROSS, форматом и способом связи; предпросмотр перед отправкой; отзыв; список со статусами и историей.

4. Архитектура

```
Браузер
  │ HTTP
  ▼
web — Next.js 16 (:3000)
  │ rewrites /api/v1/* → API_INTERNAL_URL
  ▼
api — NestJS 11 (:3001, префикс /api/v1)
  ├─► PostgreSQL 17 (Prisma 7, том, healthcheck)
  ├─► SMTP → Mailpit (локально) / внешний SMTP
  └─► MockFspProvider ◄ data/fixtures/fsp-mock.json
```

- **web** — единственная точка входа браузера; запросы `/api/v1/*` проксируются во внутренний API, поэтому cookie и CSRF работают в одном origin.
- **api** — модульный монолит: модули `auth`, `users`, `catalogs`, `candidate-profiles`, `assessments`, `companies`, `hiring-needs`, `matching`, `invitations`, `fsp`, `profile-view` / `profile-pdf`, `audit`, `mail`, `health`; **чистая доменная логика** (`src/domain/*`) отдельно от Nest и БД — её используют API, seed и скрипт валидации, поэтому показатели считаются тем же кодом, что работает в приложении.
- **PostgreSQL** — единственное хранилище состояния; ограничения целостности — в схеме и явной SQL-миграции.
- Сквозное: единый формат ошибок `{ code, message, details?, requestId }`, идентификатор запроса, подменяемые часы (для тестов), версионируемые политики.

Модель данных

Группа	Таблицы
Доступ	`User`, `Session` (хеш токена), `EmailVerificationToken`, `ConsentRecord`
Справочники	`Specialization`, `CompetencyGroup`, `Skill`
Кандидат	`CandidateProfile`, `CandidateSkill`, `CandidateQualification`, `QualificationHistory`
Тестирование	`AssessmentAttempt`, `AssessmentItem` (публичный снимок + закрытый ключ), `AssessmentAnswer` (ревизии)
Работодатель	`Company`, `HiringNeed`, `SelectionRun`, `SelectionResult`, `CandidateBookmark`
Приглашения	`Invitation`, `InvitationEvent`, `ContactAccessGrant`
ФСП и аудит	`FspLink`, `FspAchievement`, `FspLinkCodeRedemption`, `AuditEvent`

Инварианты в БД (частичные уникальные индексы и `CHECK`): не более одной активной попытки у кандидата; не более одного активного приглашения от компании кандидату; не более одного действующего согласия каждого типа; корректные зарплатные вилки; переходы статусов — условным `UPDATE … WHERE status IN (…)`, поэтому из двух конкурентных `accept` / `withdraw` выигрывает один. Подробнее — `docs/architecture.md`.

5. Механика тестирования и её устойчивость

5.1. Банк и генерация

Для каждой специализации определены три группы компетенций и по два **семейства** на группу — всего **24 семейства**, **72 пары «семейство × грейд»**. Семейство — программа, порождающая экземпляр детерминированно из `(семейство, версия, грейд, seed)` генератором sfc32: меняются числа, данные, порядок вариантов, набор правил и «ловушки». Грейд меняет **структуру** задания (например, в `fe.event-loop` Junior — синхронный код и `setTimeout(0)`, Middle — `async/await` и цепочки `then`, Senior — вложенное планирование микро- и макрозадач). Форматы частей: `SINGLE_CHOICE`, `MULTI_SELECT`, `NUMERIC`, `MATCHING`, `ORDERING`; каждая часть оценивается 0 или 1 (мульти-выбор — точное множество, число — с допуском).

Специализация	Группы компетенций
Frontend	JavaScript/TypeScript и асинхронность · Состояние и поведение интерфейса · HTTP/API и обработка ошибок клиента
Backend	API и обработка запросов · SQL и моделирование данных · Транзакции, конкурентность и надёжность
ML-инженер	Подготовка данных и утечки · Метрики и оценка моделей · Валидация, выбор модели и эксплуатационные сценарии
Manual QA	Анализ требований · Границы и классы эквивалентности · Тест-кейсы, дефекты и приоритизация

Тест — **6 мини-кейсов (по 2 на группу), 30 минут, не менее 12 оцениваемых частей**. План строится из семейств, которые кандидат видел реже; семейства внутри попытки не повторяются.

5.2. Почему задания нельзя «раздать» и почему оценка объективна

1. **Уникальный экземпляр на попытку.** В симуляции 2400 попыток на пару — ни одного точного повтора экземпляра при исключении уже виденных отпечатков; минимум 127 различающихся экземпляров на пару «семейство × грейд» при 300 seed (Т3-минимум 30). Условие, данные и порядок вариантов у разных кандидатов различаются, поэтому «ответ другого кандидата» не переносится.
2. **Закрытые ключи.** Публичная часть и ключ разделены при создании экземпляра; клиент получает только публичную часть, ключ и пояснения хранятся в БД и не попадают ни в один ответ API (проверка на каждом сгенерированном экземпляре — 0 утечек).
3. **Неизменяемый снимок.** Оценка идёт по ключу, сформированному вместе с условиями; последующие правки банка на уже выданные попытки не влияют. Семейства версионируются.
4. **Контролируемая сложность** — не обещание нейросети, а свойство генератора: структурная оценка сложности экземпляра (число шагов и ловушек) считается генератором и приводится в отчёте по каждой паре (минимум, медиана, максимум, коэффициент вариации). Полная эквивалентность сложности экземпляров не заявляется: она ограничена структурой семейства.
5. **Серверный таймер и автосохранение с ревизией** — закрытие вкладки таймер не останавливает, устаревшая запись отклоняется, повторный submit и параллельный старт безопасны.
6. **Проверка «знающего банк».** Кандидат, ответивший на каждую часть самым частым верным ответом из 400 прошлых экземпляров, набирает в среднем 28–45 баллов, максимум 72 из 100, и не проходит ни разу.

5.3. Оценка и политика грейдов

- Балл кейса — доля верных частей; балл группы — среднее кейсов × 100 (вниз); общий балл — среднее трёх групп (вниз).
- **Прохождение:** общий ≥ 75 и каждая группа ≥ 50. **«Уверенно»:** общий ≥ 90 и каждая группа ≥ 75. **Навык подтверждён:** оценён минимум в 2 разных кейсах и результат ≥ 70. Пороги — рабочие допущения, вынесенные в версионируемую политику `assessment-policy-v1`.
- **Неуспех не выдаёт и не понижает грейд.** Повтор того же уровня — не раньше чем через 24 ч после неуспеха/истечения/прерывания; более низкий грейд — сразу; новая официальная оценка подтверждённой специализации — не раньше чем через 90 дней; специализации независимы; успех на другом уровне заменяет грейд с предупреждением; высокий балл не повышает грейд автоматически — предлагается отдельный тест на следующий уровень.
- Каждое подтверждение пишется в неизменяемую историю; заявленный грейд отделён от подтверждённого, работодатель видит и фильтрует только подтверждённый.

5.4. Ограничения

Банк небольшой: набор семейств во всех попытках одной пары «специализация × грейд» совпадает (различаются условия); в 11,7 % частей верный ответ одинаков в ≥ 90 % экземпляров; тест не заменяет оценку практических навыков, нет прокторинга; тесты под конкретного работодателя не реализованы (есть предпросмотр покрытия требований).

6. Механика подбора и категоризация

6.1. Категоризация

Категория кандидата — пара «специализация × подтверждённый грейд» с подтверждёнными навыками. Самоописание (стек, опыт, роли) хранится и показывается с пометкой «указано кандидатом», но **не используется** в жёстких условиях; заявленный навык может дать лишь частичный вклад (×0,3) в желательных навыках, и только если работодатель это разрешил.

6.2. Кто виден в подборе

Подтверждён email, профиль заполнен, есть подтверждённая квалификация, активное согласие на публикацию, включена видимость, поиск не «на паузе», выбран заполненный канал раскрытия. Скрытый или снявший согласие кандидат исчезает из новых и **сохранённых** подборок и закладок.

6.3. Жёсткие условия и формула

Кандидат отсекается, если нарушено любое из: специализация, допустимый грейд, **все** обязательные навыки подтверждены тестом, формат работы, город (для не удалённой работы), пересечение зарплатных вилок (если ожидания заданы и не скрыты). **ФСП, самоописание и высокий балл не компенсируют провал обязательного условия.**

```
score = 70 · preferredFit + 25 · assessmentQuality + 5 · fspEvidence        (компоненты 0…1, итог 0…100)
preferredFit      = среднее по желательным навыкам: подтверждён тестом — 1; заявлен и разрешён — 0,3; иначе 0 (нет желательных — 1)
assessmentQuality = общий балл теста / 100
fspEvidence       = min(Σ баллов релевантных достижений / 6, 1): участие 1, призёр 2, победа 3; одно мероприятие — по сильнейшей записи
```

Порядок: `score ↓`, балл теста `↓`, `id` кандидата `↑`; сортируется всё множество, затем пагинация (порядок не зависит от размера страницы). Общий банк сортируется по `95·качество + 5·ФСП`. Score — относительная мера соответствия описанной потребности, **не вероятность найма**.

6.4. Объяснимость, сохранение и уточнение

Каждый результат содержит структурированное объяснение: пройденные условия, совпавшие и не совпавшие желательные навыки (с типом свидетельства), статус зарплаты, разложение итога по компонентам. Пример (кандидат без раскрытых контактов): «Подтверждены тестом все обязательные навыки: SQL, PostgreSQL · Ожидания 230 000–295 000 ₽ пересекаются с вилкой 250 000–350 000 ₽ · Общий балл теста: 100 из 100 · Релевантные достижения ФСП: 2 мероприятие(й), 4 балл(ов) бонуса · Что пока не подтверждено: Apache Kafka — только самоописание · Вклад в оценку соответствия 49,3 / 100». `SelectionRun` хранит снимок потребности и версию, фильтры, параметры формулы, сводку причин отсева и результаты (контакты не хранятся); уточнения создают дочернюю подборку.

6.5. Механика выхода на контакт (центр решения)

Инициатива у работодателя: потребность → подборка → карточка → приглашение. Приглашение содержит заголовок, описание, **обязательную вилку GROSS**, формат, способ связи компании и срок

ответа 14 дней; привязка к вакансии не требуется. Кандидат видит зарплату и условия **до** начала общения, а также точный список данных, которые будут раскрыты. Статусы: `SENT → VIEWED → ACCEPTED | DECLINED | WITHDRAWN | EXPIRED`; обе стороны видят актуальный статус и историю; переходы атомарны; у пары компания–кандидат — не более одного активного приглашения.

7. Приватность и безопасность

> Согласие и ролевая модель сами по себе **не доказывают** соответствие законодательству о персональных данных (152-ФЗ). Прототип работает с синтетическими данными; для реальных данных нужна отдельная организационная и техническая проверка.

- **Один презентер профиля** (`OWNER` / `EMPLOYER_REDACTED` / `EMPLOYER_GRANTED`) строит и JSON, и PDF из явного списка разрешённых полей. До принятия работодатель не видит ФИО, контакты, внешние ссылки и свободные тексты; после принятия — только выбранные каналы и только принявшая компания. Отзыв доступа немедленно скрывает контакты в карточке и PDF; уже полученное компанией технически отозвать нельзя — интерфейс об этом предупреждает.
- **Согласия** с версией, временем и отзывом; не более одного действующего каждого типа (индекс БД). Публикация — только при явном согласии и заполненном канале раскрытия.
- **Аутентификация:** Argon2id (m=19456 KiB, t=2, p=1); серверные сессии (хеш SHA-256 токена), cookie `HttpOnly`, `SameSite=Lax`, `Secure` в production; изменяющие запросы — CSRF-токен (подписанный double-submit) и проверка `Origin`.
- **Роли и изоляция:** кандидат и работодатель разграничены на API; чужие объекты — `404`; автоматический тест перебирает все 59 операций из OpenAPI.
- **Ограничение частоты** входа, регистрации, повторной отправки писем, проверки токена и привязки ФСП — в памяти процесса API (достаточно для одного экземпляра).
- **Журналы:** запросы без тел, cookie и ПДн; аудит значимых действий только с идентификаторами и статусами. Ответы API — `Cache-Control: private, no-store`. Конфигурация проверяется при старте; в production пример `SESSION_SECRET` отвергается.

8. Интеграция с реестром ФСП

> **Демонстрационная интеграция.** Реального обращения к реестру ФСП нет; контракт придуман командой и не согласован с ФСП.

- Модель платформы (`FspAchievement`) не зависит от внешнего формата; интерфейс `FspProvider { verifyLinkCode, fetchAchievements }`, реализация — `MockFspProvider`, читающий фикстуру в «внешнем» формате (`participant_id`, `one_time_code`, `results[]`) и преобразующий её во внутреннюю запись.
- **Привязка** по ФСП ID и одноразовому коду (код нельзя использовать повторно; ≤ 10 попыток за 15 минут), **синхронизация** (ошибка провайдера → `SYNC_ERROR`, последняя успешная история сохраняется), **отвязка** (удаляет достижения и бонус).
- **Показ работодателю:** без доступа — дисциплина, уровень, тип участия, место, год, навыки, релевантность; с доступом — дополнительно название, точная дата и роль в команде. Кандидат может скрыть достижения.

- **Релевантность** (`fsp-relevance-v1`): дисциплина → круг специализаций, роль в команде сужает его. В формуле — до 5 из 100.
- **Отсутствие истории:** состояния `NOT_LINKED` , `LINKED_WITHOUT_HISTORY` , `SYNC_ERROR` не ломают онбординг, подбор и карточки (проверено тестами и сквозным сценарием 5).
- **Будущее (не реализовано):** вход/привязка через ФСП ID по OIDC (Keycloak: `issuer + subject → userId`), клиент API реестра вместо фикстуры (сервис-сервис, пагинация, повторы, периодическая синхронизация), согласование контракта и справочника релевантности с экспертами ФСП, правовая основа показа достижений. Граница замены — реализация `FspProvider` .

9. Описание API

Контракт — OpenAPI 3 (`docs/openapi.json` , 100 схем); интерактивная документация — `http://localhost:3001/api/docs` (`/api/docs/openapi.json`). Все маршруты — под префиксом `/api/v1` . Для каждой операции описаны запросы, ответы и коды ошибок; реальные ответы API автоматически проверяются на соответствие контракту (Ajv, `additionalProperties: false`).

Группа	Операций	Основные маршруты
`auth`	7	`GET /auth/csrf` , `POST /auth/register` , `login` , `logout` , `verify-email` , `resend-verification` , `GET /auth/me`
`catalogs` , `health`	4 + 2	специализации, навыки, справочники, опрос; `health/live` , `health/ready`
`candidate`	6	профиль (`GET/PATCH`), предпросмотр обезличенной карточки, опрос, согласия, квалификации
`candidate-assessments`	8	допуск, старт, список, попытка, автосохранение ответа, `submit` , `abandon` , результат
`employer` , `employer-needs`	2 + 5	компания; потребности (создание, список, чтение, правка), предпросмотр плана теста
`employer-matching`	8	банк, карточка кандидата, подборка (создание, история, чтение), закладки
`employer-invitations`	4	отправка, список, чтение, отзыв
`candidate-invitations`	7	список, чтение, отметка просмотра, принятие, отказ, компании с доступом, отзыв доступа
`candidate-fsp`	4	состояние, привязка, синхронизация, отвязка
`profile-pdf`	2	PDF владельца, PDF кандидата для компании

- **Формат ошибки:** `{ "code": "…", "message": "…", "details": …, "requestId": "…" }` . Коды HTTP: `400` некорректный запрос/идентификатор, `401` нет входа, `403` запрещено для роли или email не подтверждён, `404` не найдено или недоступно, `409` конфликт состояния (повторное принятие, занятый код и др.), `422` ошибки полей, `429` ограничение частоты (с `retryAfterSeconds`), `503` не готов.
- **Идемпотентность:** старт попытки и отправка приглашения принимают `Idempotency-Key` . **Пагинация:** `page` , `pageSize` ; сортировка стабильна.
- **Аутентификация:** cookie `fsp_session` + заголовок `X-CSRF-Token` для изменяющих запросов (токен — `GET /auth/csrf`).

10. Валидация решения

Эталонных данных нет, поэтому предложена и реализована собственная процедура: `pnpm validate:quality` считает показатели по отдельным данным и формирует `docs/validation-results.{json,md}` (seed, версии правил, объёмы выборок, время запуска). Результаты ниже получены этим запуском; все данные синтетические, разметка авторская.

10.1. Подбор (протокол и итоги)

- **Данные:** 76 кандидатов (4 скрытых), 12 потребностей (по 3 на специализацию; 4 — отладочная выборка tuning, 8 — итоговая final), 228 размеченных пар «потребность–кандидат» (оценки 0/1/2 и флаг нарушения явного условия). Веса формулы заданы заранее и по разметке **не подбирались**; хэши файлов разметки фиксируются в отчёте.
- **Baseline** из Т3: те же кандидаты, отфильтрованные по специализации и допустимому грейду и отсортированные по баллу теста.
- **Метрики:** Precision@5 (делитель — 5), precision по возвращённым, nDCG@5 (линейные выигрыши), доля нарушений явных условий в top-5, покрытие, допустимые кандидаты вне выдачи; бутстреп-интервалы по потребностям.

Система (выборка final, 8 потребностей)	Precision@5	nDCG@5	Нарушения условий в top-5	Потребностей с нарушением
`matching-formula-v1`	0,68 [0,53; 0,82]	0,921 [0,867; 0,968]	0,0 %	0 из 8
baseline (специализация/грейд + балл теста)	0,65 [0,60; 0,75]	0,833 [0,698; 0,945]	36,1 %	7 из 8

Парная разница «формула − baseline»: Precision@5 +0,025 [−0,100; 0,150], nDCG@5 +0,088 [−0,055; 0,232] — **статистически не значима** при 8 потребностях. Основное преимущество даёт строгое соблюдение жёстких условий (формат, зарплатная вилка, подтверждённые навыки), которых в baseline нет. Формула теряет часть «допустимых» кандидатов (5 из 34 в final) — это следствие правил Т3. Анализ чувствительности к весам показал, что набор слабо различает варианты весов (находка F4). Автоматические проверки: скрытые кандидаты никогда не попадают в выдачу (4 из 4 исключены), порядок не зависит от перестановки входа (0 расхождений в 60 перестановках), в top-5 нет кандидатов с нарушением явного условия (0 из 47).

10.2. Тестирование и банк

Проверка	Результат
Генерация: ни одного некорректного экземпляра	0 ошибок на 14 400 экземплярах (200 seed × 72 пары)
Не менее 30 различающихся экземпляров на пару	минимум 108 при 200 seed (127 при 300)
Верный ответ даёт полный балл, пустой — 0, заведомо неверный — 0; «выбрать всё» не даёт баллов	0 нарушений
Публичная часть не содержит ключей и пояснений; показанный порядок `ORDERING` не равен верному	0 утечек, 0 совпадений
Ручные контрольные примеры, решённые человеком по одному на каждую пару «семейство × грейд»	72 из 72 проходят (проб: 1298)

Проверка	Результат
Воспроизводимость экземпляров и попыток	0 расхождений (2880 экземпляров, 240 попыток)
Верные ответы проходят порог и подтверждаются как «уверенные»	во всех 12 ячейках «специализация × грейд», 300 попыток на ячейку
Неверные и пустые ответы не проходят; «выбрать всё» — максимум 47 баллов	доля прохождений 0
Случайное угадывание: доля прохождений	≤ 1 % (максимум по ячейкам — 0)
«Знающий банк» (самый частый верный ответ из 400 экземпляров)	средний балл 28–45, максимум 72, прохождений 0 %
Профили симуляции: слабый (35 %) / сильный (95 %) / сильный со слабой группой	прохождение ≤ 0,3 % / 99,3–100 % / 8–32 %; согласованность pass/fail при повторе 0,999 / 0,994 / 0,684
Повторные попытки: точные повторы экземпляров с исключением виденных	0 из 2400 на пару

Ручные контрольные примеры выявили реальные дефекты банка (F1–F3: ошибочная формула «наименьшего числа тестов», неоднозначный приоритет, неточная формулировка) — они исправлены, семейства обновлены до v2 и решены заново.

10.3. Что валидация не показывает

- Синтетические данные и авторская разметка проверяют внутреннюю согласованность и реализованные правила, но **не доказывают**, что подтверждённый грейд соответствует реальной квалификации человека.
- Пороги (75/50/90/75; ≥ 2 кейса и ≥ 70) — рабочие допущения; симулируемые «решатели» — модель, а не люди.
- Выборка подбора мала (8 итоговых потребностей); интервалы широкие; разметку делали те же люди, что реализовали формулу.
- Для выводов о точности нужны пилот с реальными кандидатами, независимая экспертная оценка и расширенный банк.

11. Проверка качества кода и результаты тестов

Проверка	Команда	Результат
Lint	`pnpm lint`	без ошибок (api, web, contracts, e2e)
Типы	`pnpm typecheck`	без ошибок
Сборка	`pnpm build`	contracts, api, web
Unit	`pnpm test`	14 наборов, 319 тестов: границы 74/75, пороги групп, подтверждение навыков, допуск и сроки, генерация и воспроизводимость, ранжирование и фильтры, презентер профиля, автомат статусов, конфигурация, повтор синхронизации справочников при недоступной базе

Проверка	Команда	Результат
Интеграционные (реальная PostgreSQL)	`pnpm test:integration`	8 наборов, 150 тестов: авторизация и CSRF, жизненный цикл попытки и конкурентность, подбор и изоляция, приглашения (конкурентные accept/withdraw, TTL, PDF), приватность, ФСП и нестабильный провайдер, контроль доступа по всем маршрутам, контракт OpenAPI
Сквозные (Chromium)	`pnpm test:e2e`	5 из 5: (1) регистрация → подтверждение через Mailpit → опрос → профиль → тест → категория → публикация; (2) потребность → подборка → объяснение → приглашение; (3) принятие: контакты только у первой компании, отзыв доступа; (4) неуспех → нет грейда и публикации → более низкий грейд; (5) кандидат без ФСП
Docker Compose	`docker compose --profile local up -d --build`	миграции, seed, healthcheck; те же 5 E2E против контейнерного стека в production-режиме; данные переживают перезапуск
Нагрузка	`pnpm load:test`	1016 кандидатов, 20 параллельных виртуальных пользователей без пауз: p95 всех операций ≤ 0,78 с (ранжирование — 0,78, PDF — 0,53), 0 ошибок; ~52 запросов/с на одной реплике API (одна машина, синтетические данные); при 30 пользователях p95 ранжирования 1,1 с, при 60 — 2,2 с

Правильные ответы в E2E берутся из БД стенда через тестовый помощник; в API такой конечной точки нет. Защитные механизмы (CSRF, ограничение частоты, права) в тестах не отключаются. Браузерные проверки выполнены только в Chromium; мобильные экраны не проверялись.

12. Используемые библиотеки и компоненты

Версии зафиксированы точно (`package.json`, `pnpm-lock.yaml`). Установка — `pnpm install --frozen-lockfile`.

Среда и инфраструктура

Компонент	Версия	Назначение
Node.js	24.x (`⩾24.11 <25`; образ `node:24.21.0-bookworm-slim`)	среда выполнения
pnpm	10.34.6	менеджер пакетов (workspaces)
PostgreSQL	17.9 (`postgres:17.9-alpine3.23`)	СУБД
Mailpit	v1.31.4 (`axllent/mailpit`)	локальная почта (только профиль `local`)
Docker / Compose	Compose v2	запуск (проверено на Docker 29.6)

Backend (`apps/api`)

Библиотека	Версия	Назначение
`@nestjs/common`, `@nestjs/core`, `@nestjs/platform-express`	11.2.7	каркас API (модульный монолит)

Библиотека	Версия	Назначение
`@nestjs/swagger`	11.4.7	OpenAPI/Swagger
`@prisma/client`, `prisma`, `@prisma/adapter-pg`	7.10.0	ORM и миграции
`pg`	8.23.1	драйвер PostgreSQL
`express`, `helmet`, `cookie-parser`	5.2.1, 8.3.0, 1.4.7	HTTP, заголовки безопасности, cookie
`argon2`	0.45.1	хеширование паролей (Argon2id)
`class-validator`, `class-transformer`, `zod`	0.15.1, 0.5.1, 4.6.5	валидация DTO и конфигурации
`nodemailer`	10.0.14	отправка писем
`pdfkit`	0.20.2	PDF-профиль
`dotenv`, `reflect-metadata`, `rxjs`	18.0.5, 0.2.2, 7.8.2	конфигурация и служебные зависимости Nest
TypeScript	5.9.3	язык
`@nestjs/cli`, `@swc/core`, `@swc/jest`, `jest`, `supertest`, `@nestjs/testing`	11.0.24, 1.16.13, 0.2.39, 30.5.2, 7.3.1, 11.2.7	сборка и тесты
`ajv`, `ts-json-schema-generator`, `pdfjs-dist`, `tsx`	8.20.0, 2.9.0, 6.4.299, 4.23.15	проверка контракта, чтение PDF в тестах, запуск скриптов
`eslint`, `typescript-eslint`	9.39.5, 8.71.0	статический анализ

Frontend (`apps/web`)

Библиотека	Версия	Назначение
`next`, `react`, `react-dom`	16.3.8, 19.3.0, 19.3.0	веб-приложение (App Router, standalone)
`@tanstack/react-query`	5.104.1	запросы и кэш
`react-hook-form`, `@hookform/resolvers`, `zod`	7.89.0, 5.9.1, 4.6.5	формы и валидация
`tailwindcss`, `@tailwindcss/postcss`	4.3.3	стили
`clsx`	2.1.1	утилита классов
JetBrains Mono	OFL, локально	фирменный шрифт (без внешних запросов)

Сквозные тесты и инструменты документации

Библиотека	Версия	Назначение
`@playwright/test`	1.63.0	сквозные тесты и печать PDF
`pg`	8.23.1	помощник тестов (чтение ключей стенда)
`docx`, `marked`, `jszip`	9.8.1, 18.1.0, 3.10.1	экспорт документации в DOCX/PDF

Лицензии

Прямые зависимости — открытые лицензии (MIT, Apache-2.0, ISC, BSD). По отчёту `pnpm licenses list --prod`: MIT — 329 пакетов, Apache-2.0 — 42, ISC — 34, BSD — 13; исключения среди

транзитивных: LGPL-3.0 (бинарные сборки libvips в `sharp`, используются Next.js для оптимизации изображений), EPL-2.0 (`elkjs`, дерево Prisma CLI), CC-BY-4.0 (данные `caniuse-lite`), Python-2.0 (`argparse`), Unlicense, MIT-0. Лицензия самого кода проекта владельцем не выбрана. Код предоставляется открытым, без обфускации.

13. Сборка, развёртывание и локальный запуск

Требования: Node.js 24, pnpm 10.34.6 (`corepack enable`), Docker с Compose v2. Подробнее и с переменными окружения — `README.md`.

Вариант A. На хосте (PostgreSQL и почта в Docker)

```
pnpm install
cp .env.example .env            # PowerShell: Copy-Item .env.example .env
pnpm infra:up                   # PostgreSQL 127.0.0.1:5433, Mailpit (SMTP 1025, UI
http://localhost:8025)
pnpm db:migrate                 # prisma migrate deploy
pnpm db:seed                    # демонстрационные данные
pnpm dev                        # web http://localhost:3000, API http://localhost:3001
```

Продуктовая сборка: `pnpm build && pnpm start`.

Вариант B. Всё в контейнерах

```
cp .env.example .env
# замените SESSION_SECRET случайным значением: openssl rand -hex 32
docker compose --profile local up -d --build   # postgres → migrate → api → web, mailpit
docker compose --profile seed run --rm seed    # демонстрационные данные
```

Web — `http://localhost:3000`, письма — `http://localhost:8025`, API-документация — `http://localhost:3001/api/docs`. Данные хранятся в томе `postgres-data`; миграции выполняет одноразовый сервис `migrate` до старта API (`db push` не используется); у postgres, api и web есть healthcheck. Порты postgres, mailpit и API привязаны к `127.0.0.1`; интерфейс Mailpit нельзя выставлять публично. Внешний запуск требует настоящего SMTP (`DOCKER_SMTP_HOST`, `SMTP_*`) и HTTPS-прокси (`SESSION_COOKIE_SECURE=true`, `APP_ORIGIN=https://…`). Публикация не выполнялась.

Проверки и сопутствующие команды

`pnpm lint`, `pnpm typecheck`, `pnpm test`, `pnpm test:integration` (нужен `pnpm infra:up`), `pnpm test:e2e` (нужны запущенный стенд и `pnpm --filter @fsp/e2e install-browsers`), `pnpm validate:quality`, `pnpm bank:manifest`, `pnpm openapi:export`, `pnpm docs:export` (PDF и DOCX этого документа), `pnpm load:test`. Конфигурация проверяется при старте с понятной ошибкой.

14. Демонстрация

Сценарий на 5–7 минут — `docs/demo-script.md`. Пароль демо-аккаунтов — `Demo-Password-2026`. Основные аккаунты:

Роль	Логин	Особенность
Кандидат	`anna.demidova@demo.example`	Backend Middle (балл 83), привязан ФСП, входящее приглашение от «Демо Финтех»
Кандидат	`ready.candidate@demo.example`, `failed.candidate@demo.example`, `cooldown.candidate@demo.example`	тест не пройден ещё; неуспех Middle (пересдача закрыта, Junior доступен); неуспех Senior 3 часа назад
Кандидат	`{fe,be,ml,qa}.{junior,middle,senior}.{01…08}@demo.example`	96 основных кандидатов с подтверждёнными категориями
Работодатель	`hr.fintech@…`, `hr.retail@…`, `hr.industry@…`, `hr.edu@…` (`@demo.example`)	четыре вымышленные компании, 9 потребностей

Seed создаёт 114 кандидатов (96 основных и 18 особых состояний: нет теста, неудача, задержка пересдачи, прерванная и истёкшая попытки, скрытый, без согласия, пауза, неподтверждённый email, срок переоценки, сбой и пустая история ФСП), 4 компании, 9 потребностей, 10 приглашений во всех статусах, 3 доступа к контактам (один из них отозван), 5 сохранённых подборок, 5 закладок.

15. Допущения и вопросы к организаторам

Решения команды, не вытекающие из ТЗ (подробно — `docs/assumptions.md`): одна активная специализация в интерфейсе (модель допускает несколько); пороги 75/50/90/75; 24 ч и 90 дней; тест общий для специализации и грейда; обязательный навык — только подтверждаемый заданиями; формула 70/25/5; бонус ФСП 1/2/3, нормировка 6; срок приглашения 14 дней; вход по email и паролю; зарплата — рубли в месяц до вычета налогов. Приоритетные вопросы организаторам: допустимые шкалы и пороги; общий или «под работодателя» тест; формат доступа к реестру ФСП; тексты согласий и сроки хранения; проверка компаний; нужны ли отклики, ATS, мобильная версия. Места изменения решений указаны в таблице раздела 5 `docs/assumptions.md` (политики версионируются, интерфейс и схема БД менять не нужно).

16. Ограничения и что вне MVP

- Данные синтетические; соответствие 152-ФЗ не заявляется; тексты согласий — рабочие формулировки; удаление и сроки хранения данных не реализованы.
- Пороги и банк не откалиброваны на реальных людях; валидация — на синтетической авторской разметке; разница с baseline по Precision@5 не значима статистически.
- ФСП — только демонстрационный провайдер; реальный API и OIDC отсутствуют.
- Не реализованы: загрузка PDF-резюме, регулярные задания, вакансии и отклики, проверка компаний, ATS, восстановление пароля, несколько сотрудников в компании, мобильная адаптация.
- Ограничение частоты — в памяти одного процесса; ранжирование — в backend после SQL-фильтрации; одна реплика API выдерживает ~50 запросов/с при 1000 кандидатов (при 20 параллельных пользователях p95 ≤ 0,78 с, при 60 — до 2,2 с).

- Проверки только в Chromium; Docker-запуск проверен на машине разработки; полностью «чистая машина» не воспроизводилась.
- Презентация (pptx/pdf), видео и публикация внешнего стенда — действия владельца проекта; публикация не выполнялась.

17. Развитие (концепции, не реализованный функционал)

Раздел описывает направления и архитектурный подход, который можно доработать совместно с ФСП. Ничего из перечисленного ниже в MVP не реализовано.

17.1. Проверка компаний и защита от фиктивных предложений

Угрозы: фиктивные компании и «вакансии» для сбора контактов, заманивание в платные курсы и схемы, завышенные или обманчивые вилки, массовые шаблонные приглашения, выманивание персональных данных.

Подход — эшелонированная защита:

1. *Верификация организации.* Статусы `UNVERIFIED → EMAIL_DOMAIN_VERIFIED → LEGAL_ENTITY_VERIFIED`: подтверждение корпоративного домена (письмо на домен сайта, SPF/DKIM) и проверка юридического лица (ИНН/ОГРН по открытому реестру). Приглашения от неверифицированных компаний — с пометкой для кандидата и строгими квотами.
2. *Лимиты и сигналы.* Суточные квоты приглашений на компанию, дедупликация шаблонных текстов, доля принятий/отказов/жалоб как показатель доверия, проверка вилки на аномалии относительно рынка для специализации и грейда, ограничение повторных приглашений после отказа (сейчас ограничен только один активный).
3. *Жалобы и модерация.* Кнопка «Пожаловаться» в приглашении, очередь модерации, действия (предупреждение, ограничение, блокировка), апелляция, журнал решений.
4. *Прозрачность для кандидата.* Бейдж верификации и краткая история компании (сколько приглашений отправлено/принято) в карточке приглашения.

Как встроить: поля `Company.verificationStatus`, таблицы `CompanyVerification`, `Report`, `InvitationQuota`; проверка квот и статуса в `InvitationsService.send` перед созданием записи; роль `MODERATOR`; события аудита. Идемпотентность, атомарные переходы и аудит приглашений, которые нужны для этого, уже есть.

17.2. Интеграция с ATS работодателей

Цель: работодатель ведёт кандидатов и статусы в своей ATS, а платформа остаётся источником подтверждённых кандидатов.

- *Исходящие события (webhooks)* `invitation.sent|viewed|accepted|declined|withdrawn|expired`, `contact_access.granted|revoked`: до принятия — обезличенный payload, после — только раскрытые каналы; подпись HMAC, ключи на компанию, повторы с экспоненциальной задержкой, журнал доставок.
- *Входящие данные:* создание потребностей из вакансий ATS, обратные статусы найма (нанят/отказ) как сигнал для будущей калибровки подбора.

- *Адаптеры:* интерфейс `AtsAdapter` по аналогии с `FspProvider`; конкретные коннекторы — отдельные модули. Авторизация — OAuth2 (client credentials), scopes по компании.
- *Правовая сторона:* передача данных кандидата в ATS работодателя — отдельное явное согласие кандидата.

17.3. Вакансии и отклики (дополнительный сценарий ТЗ)

Модели `Vacancy` (публичная форма потребности с обязательной вилкой, редактирование, закрытие) и `Application` (`SENT → VIEWED → ACCEPTED | DECLINED`). Отклик — это явное согласие на раскрытие выбранных контактов; зарплата обязательна; откликнувшиеся ранжируются той же формулой и объясняются так же. Кабинеты: кандидату — список вакансий и «мои отклики», работодателю — вакансии и входящие отклики. Основная механика приглашений остаётся главной.

17.4. Регулярные короткие задания

Работодатель создаёт короткое задание (из семейств банка или собственное); платформа периодически предлагает его кандидатам подходящей категории; результат влияет на **актуальность профиля** (сигнал свежести с затуханием), но **не** меняет подтверждённый грейд. Ограничения: добровольность, частота (не чаще N раз в месяц), серверный таймер и уникализация экземпляров; структурированные задания проверяются автоматически, свободные («предложите подход») — вручную работодателем.

17.5. Мобильная адаптация

Приоритетные экраны — приглашения, подборка, карточка кандидата; прохождение теста с таймером и автосохранением на мобильном; адаптивные сетки и тач-цели, проверка доступности и контраста; уведомления о приглашениях (email/push).

17.6. ФСП ID и OIDC (Keycloak)

План — раздел 8: вход/привязка по OIDC, клиент API реестра, периодическая синхронизация, согласование контракта и справочника релевантности.

17.7. Масштабирование и эксплуатация

Общий счётчик ограничения частоты (PostgreSQL/Redis) и несколько реплик API; очередь исходящих писем с повтором (outbox); кэш справочников; предрасчёт и индексы для подбора при росте базы; метрики, централизованные логи, алертинг; резервное копирование БД; миграции и проверки в CI.

17.8. Качество оценки

Пилот с реальными кандидатами; психометрический анализ (дискриминативность и надёжность заданий по реальным ответам, стабильность грейда при повторных измерениях); экспертная калибровка порогов; расширение банка (в разы больше семейств, чтобы набор тем менялся между попытками); независимая разметка подбора и сравнение версий формулы по реальным исходам (принятия приглашений).

18. Организация работы двух разработчиков

Зона	Разработчик 1: кандидат + тестирование	Разработчик 2: работодатель + подбор + приглашения
API	`candidate-profiles`, `assessments`, `fsp`, `catalogs` (опрос)	`companies`, `hiring-needs`, `matching`, `invitations`, `profile-pdf`
Домен	`domain/assessment/*`, `domain/candidate/*`, `domain/catalog/*`	`domain/matching/*`, `domain/invitations/*`
Интерфейс	`apps/web/src/app/candidate/*`	`apps/web/src/app/employer/*`, `components/employer/*`
Проверки	`assessments`, `fsp`, банк и контрольные примеры, E2E 1, 4, 5	`matching`, `invitations`, `privacy`, E2E 2, 3

Общие и согласуемые в коде части: схема БД и миграции, контракт `packages/contracts`, авторизация и сессии, единый презентер профиля (`domain/privacy/profile-view.ts`), политики (`config/policies.ts`), OpenAPI. Две несовместимые реализации общих сущностей не создавались.

19. Состав документации

Документ	Содержание
`README.md`	запуск, переменные окружения, команды, учётные записи, ограничения
`docs/architecture.md`	компоненты, модули, модель данных, потоки
`docs/assessment.md`	банк, оценка, политика грейдов, защита от повторов
`docs/matching.md`	фильтры, формула, объяснения, baseline
`docs/privacy.md`	матрица доступа, PDF, доступ к контактам, журналы
`docs/fsp-integration.md`	mock-контракт, будущая интеграция
`docs/assumptions.md`	допущения, вопросы к организаторам, места изменений
`docs/validation-results.md` / `.json`	результаты `pnpm validate:quality`
`docs/requirements-matrix.md`	требования → функции, экраны, проверки
`docs/implementation-status.md`	статус, замеры, ограничения, находки
`docs/demo-script.md`, `docs/presentation-outline.md`	демонстрация и план презентации
`docs/openapi.json`	контракт API